

Reviewer Pack — Minimal Rank of Brauer Groups vs Resolution Refutation Depth

Entry 0f2b3bc2896f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 04:03:46 UTC

Minimal Rank of Brauer Groups vs Resolution Refutation Depth

Entry ID: 0f2b3bc2896f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 04:03:46 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Group) **Field B** (complexity object): Complexity Theory: Resolution Refutation Complexity

Statement:

[‘For every n -variable CNF formula F , the minimal rank of the Brauer group of its associated vector space over the finite field with two elements is $\Omega(2^{n/4})$.’, ‘This lower bound holds for all instances F with at least one satisfying assignment.’, ‘The conjecture is falsified if there exists an instance F such that the minimal rank of its Brauer group is $O(n^{1/8})$.’]

Rationale (proposer’s reasoning):

[‘The Brauer group captures non-trivial linear algebraic invariants, which may reveal hidden complexity in the structure of CNF formulas.’, ‘This conjecture explores whether the depth of resolution refutations can be characterized by representation-theoretic properties.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f5c1eb38ef695d2d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all $n \leq 40$, the correlation coefficient between minimal Brauer group rank $\rho(\text{Br}(V))$ and resolution refutation depth $d(F)$ across 30 seeds is ≥ 0.5 AND $\rho(\text{Br}(V))$ meets $\Omega(2^{n/4})$. It is falsified if any seed produces a $\rho(\text{Br}(V)) \leq n^{1/8}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Brauer Group' AND 'Resolution refutation complexity' - 'Minimal rank' AND 'Brauer group' AND 'CNF formula' - 'Complexity theory' AND 'finite field' AND 'minimal rank Brauer group'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(random.randint(1, n))]
            if all(abs(x) != abs(y) for x, y in itertools.combinations(clause, 2)):
                clauses.append(clause)
        return clauses

    def cnf_to_vector_space(cnf):
        n = max(abs(var) for clause in cnf for var in clause)
        V = [[0] * (n + 1) for _ in range(2**n)]
        for i, clause in enumerate(cnf):
            for var in clause:
                if var > 0:
                    V[i][var] = 1
                else:
                    V[i][-var] = -1
        return V

    def gaussian_elimination(V):
        m, n = len(V), len(V[0])
        rank = 0
        for j in range(n):
            i_max = next((i for i in range(rank, m) if V[i][j]), None)
            if i_max is not None:
                V[rank], V[i_max] = V[i_max], V[rank]
```

```

        for i in range(rank + 1, m):
            factor = Fraction(V[i][j], V[rank][j])
            for k in range(n):
                V[i][k] -= factor * V[rank][k]
            rank += 1
    return rank

def bruer_group_rank(V):
    return gaussian_elimination(V)

def resolution_refutation_depth(cnf):
    n = max(abs(var) for clause in cnf for var in clause)
    stack = []
    for clause in cnf:
        if all(x not in stack and -x not in stack for x in clause):
            stack.extend(clause)
        else:
            new_clause = [x for x in clause if x not in stack and -x not in stack]
            if not new_clause:
                return 0
            stack.append(new_clause[0])
    return len(stack)

n = random.randint(5, 40)
cnf = generate_cnf(n)
V = cnf_to_vector_space(cnf)
rho = bruer_group_rank(V)
d = resolution_refutation_depth(cnf)

conjecture_holds = rho >= 2**(n/4) and rho > n**1/8
counterexample = "" if conjecture_holds else "rho <= n^1/8"

return {
    "metric_name": "Brauer Group Rank vs Resolution Refutation Depth",
    "metric_value": rho,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")

```

```

elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"rho <= n^1/8\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the results from this test. | next: Run the test again with increased time limits to ensure it completes and produces valid results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11719
2	propose	ollama_remote	glm4:latest	0	0	11033
3	propose	ollama_remote	glm4:latest	0	0	12464
4	preregistration	ollama_remote	glm4:latest	0	0	6064
5	novelty	ollama_remote	glm4:latest	0	0	4718
6	novelty	ollama_remote	glm4:latest	0	0	7644
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13709
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9962
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27571
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13406
11	judge	ollama_remote	glm4:latest	0	0	45754

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 164043 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/0f2b3bc2896f.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f2b3bc2896f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0f2b3bc2896f.tar.gz` (if generated)